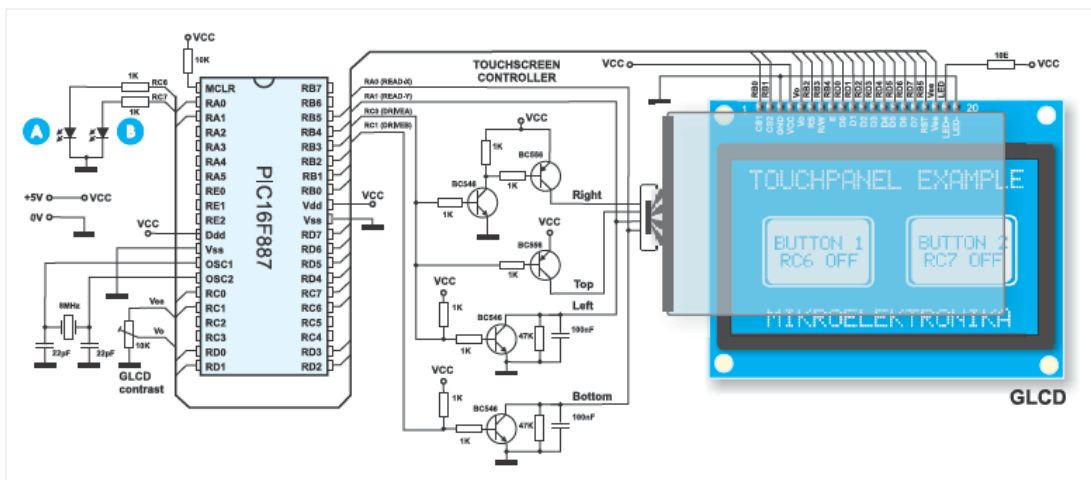


4.17 EXAMPLE 15

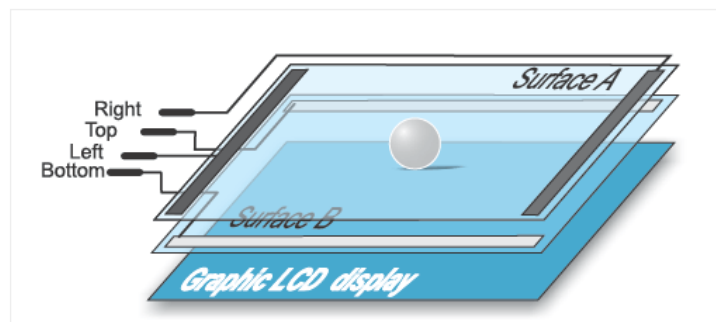
MIKROELEKTRONIKA

Using touch panel...

A touch panel is a thin, self-adhesive transparent panel placed over the screen of a graphic LCD. It is very sensitive to pressure so that even a soft touch causes some changes on output signal. There are a few types of touch panel. The simplest one is the resistive touch panel.



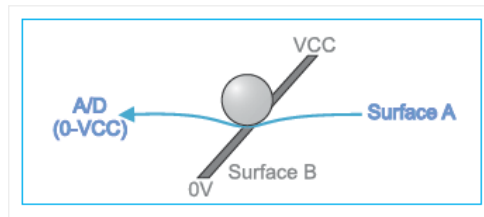
It consists of two transparent rigid foils, forming a 'sandwich' structure, that have resistive layers on their inner sides. The resistance of these layers usually does not exceed 1K. The opposite sides of the foils have contacts available for use through a flat cable.



The process of determining coordinates of the point in which the touch panel is pressed can be broken up into two steps. The first one is the determination of the X coordinate and the second one is the determination of the Y coordinate of the point.

In order to determine the X coordinate, it is necessary to connect the left contact on the *surface A* to ground and the right contact to the power supply. This enables a voltage divider to be obtained by pressing the touch panel. The value of the divider is read on the bottom contact of the *surface B*. Voltage can be in the range of 0V to the power supply and depends on the X coordinate. If the point is closer to the left contact of the surface A, the

voltage will be closer to 0V.



In order to determine the Y coordinate, it is necessary to connect the bottom contact on the *surface B* to ground, and the upper contact to power supply. In this case, the voltage is read on the left contact of the *surface A*.

In order to connect a touch panel to the microcontroller it is necessary to create a circuit for touch panel control. By means of this circuit, the microcontroller connects appropriate contacts of the touch panel to ground and the power supply (as described above) in order to determine the X and Y coordinates. The bottom contact of the *surface B* and left contact of the *surface A* are connected to the microcontroller's A/D converter. The X and Y coordinates are determined by measuring voltage on these contacts, respectively. The software consists of writing a menu on graphic LCD, turning the circuit for touch panel control on/off (driving touch panel) and reading the values of A/D converter which actually represent the X and Y coordinates of the point.

Once the coordinates are determined, it is possible to decide what we want the microcontroller to do. In this example, microcontroller turns on/off two digital pins, connected to LED diodes A and B.

This example use functions belonging to the *Glcd* and *ADC* library.

Considering that the touch panel surface is slightly larger than the surface of the graphic LCD, in case you want greater accuracy when determining the coordinates, it is necessary to perform the software calibration of the touch panel.

```

1  const char msg1[] = "TOUCHPANEL EXAMPLE";
2  const char msg2[] = "MIKROELEKTRONIKA";
3  const char msg3[] = "BUTTON1";
4  const char msg4[] = "BUTTON2";
5  const char msg5[] = "RC6 OFF";
6  const char msg6[] = "RC7 OFF";
7  const char msg7[] = "RC6 ON ";
8  const char msg8[] = "RC7 ON ";
9
10 long x_coord, y_coord, x_coord128, y_coord64; // scaled x-y position
11 char msg[16];
12
13 char * CopyConst2Ram(char * dest, const char * src){
14     for(;*dest++ = *src++;)
15     ;
16     return dest;
17 }
18
19 // Glcd module connections
20 char GLCD_DataPort at PORTD;
21
22 sbit GLCD_CS1 at RB0_bit;
23 sbit GLCD_CS2 at RB1_bit;
24 sbit GLCD_RS at RB2_bit;
25 sbit GLCD_RW at RB3_bit;
26 sbit GLCD_EN at RB4_bit;
27 sbit GLCD_RST at RB5_bit;
28
29 sbit GLCD_CS1_Direction at TRISB0_bit;
30 sbit GLCD_CS2_Direction at TRISB1_bit;
31 sbit GLCD_RS_Direction at TRISB2_bit;
32 sbit GLCD_RW_Direction at TRISB3_bit;
33 sbit GLCD_EN_Direction at TRISB4_bit;
34 sbit GLCD_RST_Direction at TRISB5_bit;
35 // End Glcd module connections
36
37 unsigned int GetX() {
38     //reading X
39     PORTC.F0 = 1; // DRIVEA = 1 (LEFT drive on, RIGHT drive on
40     // , TOP drive off )
41     PORTC.F1 = 0; // DRIVEB = 0 (BOTTOM drive off )
42     Delay_ms(5);
43     return ADC_read(0); // reading X value from RA0 (BOTTOM)
44 }
45
46 unsigned int GetY() {
47     //reading Y
48     PORTC.F0 = 0; // DRIVEA = 0 (LEFT drive off , RIGHT drive off
49     // , TOP drive on)
50     PORTC.F1 = 1; // DRIVEB = 1 (BOTTOM drive on)
51     Delay_ms(5);

```

```

52  return ADC_read(1); // reading Y value from RA1 (from LEFT)
53  }
54
55  void main() {
56  PORTA = 0x00;
57  TRISA = 0x03; // RA0 i RA1 are analog inputs
58  ANSEL = 0x03;
59  ANSELH = 0; // Configure other AN pins as digital I/O
60  PORTC = 0;
61  TRISC = 0; // PORTC is output
62  Glcd_Init(); // Glcd_Init_EP5
63  Glcd_Set_Font(FontSystem5x7_v2, 5, 7, 32); // Choose font size 5x7
64  Glcd_Fill(0); // Clear GLCD
65  CopyConst2Ram(msg,msg1); // Copy "TOUCHPANEL EXAMPLE" string to RAM
66  Glcd_Write_Text(msg,10,0,1);
67  CopyConst2Ram(msg,msg2); // Copy "MIKROELEKTRONIKA" string to RAM
68  Glcd_Write_Text(msg,17,7,1);
69
70  //Display Buttons on GLCD:
71  Glcd_Rectangle(8,16,60,48,1);
72  Glcd_Rectangle(68,16,120,48,1);
73  Glcd_Box(10,18,58,46,1);
74  Glcd_Box(70,18,118,46,1);
75  CopyConst2Ram(msg,msg3); // Copy "BUTTON1" string to RAM
76  Glcd_Write_Text(msg,14,3,0);
77  CopyConst2Ram(msg,msg5); // Copy "RC6 OFF" string to RAM
78  Glcd_Write_Text(msg,14,4,0);
79  CopyConst2Ram(msg,msg4); // Copy "BUTTON2" string to RAM
80  Glcd_Write_Text(msg,74,3,0);
81  CopyConst2Ram(msg,msg6); // Copy "RC7 OFF" string to RAM
82  Glcd_Write_Text(msg,74,4,0);
83
84  while (1) {
85  // read X-Y and convert it to 128x64 space
86  x_coord = GetX();
87  y_coord = GetY();
88  x_coord128 = (x_coord * 128) / 1024;
89  y_coord64 = 64 - ((y_coord * 64) / 1024);
90
91  //if BUTTON1 is selected
92  if ((x_coord128 >= 10) && (x_coord128 <= 58) && (y_coord64 >= 18) && (y_coord64 <= 46))
93  if(PORTC.F6 == 0) {
94  PORTC.F6 = 1;
95  CopyConst2Ram(msg,msg7); // Copy "RC6 ON " string to RAM
96  Glcd_Write_Text(msg,14,4,0);
97  }
98  else {
99  PORTC.F6 = 0;
100 CopyConst2Ram(msg,msg5); // Copy "RC6 OFF" string to RAM
101 Glcd_Write_Text(msg,14,4,0);
102 }
103 }
104
105 //if BUTTON2 is selected
106 if ((x_coord128 >= 70) && (x_coord128 <= 118) && (y_coord64 >= 18) && (y_coord64 <= 46))
107 if(PORTC.F7 == 0) {
108 PORTC.F7 = 1;
109 CopyConst2Ram(msg,msg8); // Copy "RC7 ON " string to RAM
110 Glcd_Write_Text(msg,74,4,0);
111 }
112 else {
113 PORTC.F7 = 0;
114 CopyConst2Ram(msg,msg6); // Copy "RC7 OFF" string to RAM
115 Glcd_Write_Text(msg,74,4,0);
116 }
117 }
118 Delay_ms(100);
119 }
120 }

```

In order to make this example work properly, it is necessary to tick off the following libraries in the *Library Manager* prior to compiling:

- GLCD
- ADC
- C_Stdlib



4.17 Example 15 by MikroElektronika is licensed under a Creative Commons Attribution 4.0 International License, except where otherwise noted.

